

Numerical Methods for Mathematical Finance

Potential Projects

Matteo Garbelli

University of Verona

January 2026

List of Potential Projects

1. Rough Volatility: Modelling and Pricing with Fractional Brownian Motion
2. Approximation of Iterated Stochastic Integrals and Rough Path Applications
3. Signature Methods for Financial Time Series Analysis and Trading
4. Neural Operators for Solving High-Dimensional PDEs in Finance
5. Deep Learning Methods for solving BSDEs
6. Entropic Risk Minimization for Multi-asset Portfolio Optimization via Neural Networks
7. Stochastic Volatility Models besides Heston Model (e.g., Local Volatility Model)
8. Simulation of Jump Processes (Lévy Processes) and Feynman-Kac theorem for jump diffusion processes
9. Energy Markets: Modeling and Pricing Carbon Credits with Jump Diffusion
10. FinTech: Reinforcement Learning for Optimal Trading Strategies

Rough Volatility: Modelling and Pricing with Fractional Brownian Motion

Objective: Implement rough volatility models driven by fractional Brownian motion (fBm) with Hurst exponent $H < 0.5$.

Mathematical Framework: Rough Bergomi model

$$dS_t = rS_t dt + \sqrt{v_t} S_t dW_t$$

$$v_t = \xi_0(t) \exp \left(\eta \tilde{W}_t - \frac{1}{2} \eta^2 t^{2H} \right)$$

where $\tilde{W}_t$ is a fractional Brownian motion with $H \approx 0.1$.

Coding Reference: Python with `stochvol` or hybrid scheme for fBm.

Paper Reference:

- Pricing under rough volatility (Gatheral, Jaisson, Rosenbaum, 2018)

Approximation of Iterated Stochastic Integrals and Rough Path Applications

Objective: Explore numerical methods for iterated stochastic integrals (Lévy area) arising in rough path theory.

Mathematical Framework:

$$\iint dB dB$$

Lyons' theory enhances Brownian motion to a rough path $(B_t, \iint dB dB)$.

Coding Reference: Python - implement Wiktorsson's algorithm for Lévy area simulation.

Paper Reference:

- Lyons, T. (1998). Differential equations driven by rough signals
- Wiktorsson, M. (2001). Joint simulation of iterated Itô integrals

Signature Methods for Financial Time Series Analysis and Trading

Objective: Use the Signature Transform from rough path theory for feature extraction in financial time series.

Mathematical Framework: The signature of a path $X = (X_t)_{t \in [0, T]}$ is a collection of all its iterated integrals, providing a faithful feature set.

Coding Reference: Python packages: `esig` and `signatory` (PyTorch).

Paper Reference:

- A Primer on the Signature Method in ML (Chevyrev & Kormilitzin, 2016)
- Signature-based Portfolio Optimisation (Horvath et al., 2021)

Neural Operators for Solving High-Dimensional PDEs in Finance

Objective: Implement Neural Operators (e.g., Fourier Neural Operators) for solving high-dimensional PDEs in finance.

Mathematical Framework: Learn the *solution operator* of a family of PDEs. Given parameters (initial condition, volatility), the operator outputs the solution.

Coding Reference: PyTorch or TensorFlow.

Repository: https://github.com/zongyi-li/fourier_neural_operator

Paper Reference:

- Fourier Neural Operator for Parametric PDEs (Li et al., 2020)
- Physics-informed DeepONets (Lu et al., 2021)

Deep Learning Methods for BSDEs

Objective: Implement deep learning models for solving BSDEs.

Mathematical Framework:

$$Y_t = \xi + \int_t^T f(s, Y_s, Z_s) ds - \int_t^T Z_s dW_s$$

Approximate the solution (Y, Z) with neural networks.

Coding Reference: PyTorch or TensorFlow

Paper Reference:

- Deep BSDEs for High-Dimensional PDEs (E et al., 2017)
- Backward Stochastic Differential Equations in Finance (El Karoui et al., 2002)

Entropic Risk Minimization for Multi-Asset Portfolio Optimization

Objective: Optimize a portfolio using entropic risk minimization.

Mathematical Framework:

$$\rho(X) = \frac{1}{\eta} \log \mathbb{E} [e^{-\eta X}]$$

Coding Reference: PyTorch or TensorFlow; PyPortfolioOpt.

Paper Reference:

- Entropic Risk Minimization and Portfolio Selection (Hirschberger et al., 2007)
- Neural Networks for Portfolio Management (Heaton et al., 2017)

Stochastic Volatility Models (Beyond Heston Model)

Objective: Implement models such as the Local Volatility Model or the SABR model.

Mathematical Framework:

$$dS_t = rS_t dt + b(V_t) \sigma_{LSV}(S_t, t) S_t dW_t$$

Coding Reference: QuantLib or custom simulations.

Paper Reference:

- A Local Volatility Model (Dupire, 1994)
- The SABR Model in Illiquid Markets (Hagan et al., 2002)

Simulation of Jump Processes (Lévy Processes)

Objective: Simulate jump-diffusion processes and price derivatives.

Mathematical Framework:

$$dX_t = \mu dt + \sigma dW_t + dJ_t$$

Coding Reference: Python packages like NumPy, SciPy, or PyMC.

Paper Reference: Option Pricing when Jump-Diffusion Processes are Used (Merton, 1976)

Energy Markets: Modeling and Pricing Carbon Credits

Objective: Model and price carbon credits with jump diffusion.

Mathematical Framework: Jump-diffusion models for carbon price dynamics.

Coding Reference: QuantLib or custom implementations.

Paper Reference: The Pricing of Emission Allowances in Market Frictions (Carmona et al., 2009)

FinTech: Reinforcement Learning for Optimal Trading Strategies

Objective: Develop RL algorithms for optimal trading.

Mathematical Framework: Formulate as a Markov Decision Process.

Coding Reference: Use RL libraries like Stable-Baselines3.

Paper Reference: Deep Reinforcement Learning in High Frequency Trading (Zhang et al., 2020)